

CHORUS

System Architecture Report

Multi-Agent Knowledge Management System
Knowledge Lab, University of Chicago

Generated: January 13, 2026
Version: 2.0

1. Executive Summary

CHORUS is a sophisticated multi-agent knowledge management and retrieval system designed for the Knowledge Lab at the University of Chicago. The system combines intent-driven query routing, hybrid retrieval-augmented generation (RAG), structured registry lookups, and conversation memory to provide intelligent responses about lab members, research projects, computing resources, and documents.

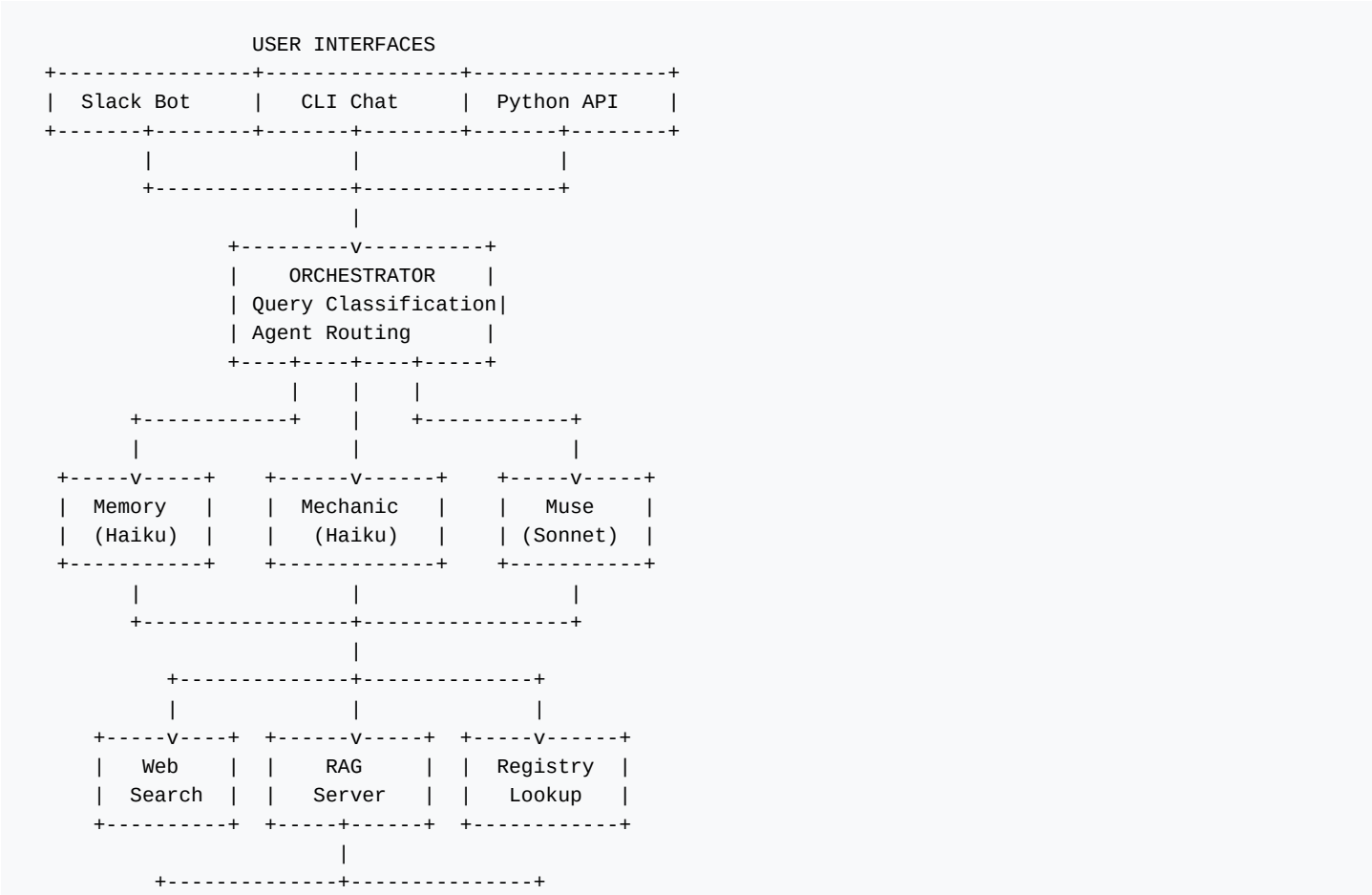
~9,000 Lines of Code	11,188 Indexed Chunks	71 Lab Members	4 Specialist Agents
---	--	-------------------------------------	--

Key Capabilities

- Multi-agent orchestration with intent classification
- Hybrid RAG (semantic + keyword + structured search)
- Anthropic built-in web search integration
- Enhanced conversation memory with entity tracking
- Multiple interfaces: Slack bot, CLI, Python API

2. System Architecture Overview

The system follows a layered architecture with user interfaces at the top, an orchestration layer for query routing, specialist agents for domain-specific processing, and a data layer combining structured registries with semantic indexes.



CHORUS System Architecture Report

+----v----+	+----v----+	+----v----+
FAISS	BM25	JSON
Semantic	Keyword	Registry
+-----+	+-----+	+-----+

3. Core Components

3.1 File Structure

Component	File	Lines	Purpose
Core Agent	core_agent.py	804	Monolithic agent
Orchestrator	orchestrator.py	488	Multi-agent router
Classifier	classifier.py	323	Intent classification
Memory Agent	memory.py	650+	Factual retrieval
RAG Server	rag_http_server.py	1,026	Hybrid search API
Slack Bot	slack_bot.py	700+	Slack integration

3.2 Specialist Agents

Agent	Intent	Model	Use Case
Memory	lookup	Haiku	Who, what, where questions
Mechanic	technical	Haiku	Setup, debugging, access
Muse	explore	Sonnet	Analysis, synthesis
Matchmaker	connect	Sonnet	Finding collaborators

4. RAG System

The RAG system uses a three-way hybrid search combining semantic embeddings (FAISS), keyword matching (BM25), and structured registry lookups. Results are merged, reranked using a cross-encoder, and diversified using MMR.

4.1 Index Statistics

Index	Size	Model/Method	Latency
FAISS	11,188 chunks	MiniLM-L6-v2 (384d)	<1ms
BM25	11,188 chunks	rank-bm25	<1ms
Registry	71 people, 10 datasets	JSON lookup	<1ms

5. Data Sources & Registries

5.1 Lab Registry

71 Lab Members	482 Publications	3 Projects	10+ Datasets
--------------------------	----------------------------	----------------------	------------------------

5.2 Midway Registry

Category	Count	Examples
Data Snapshots	12	OpenAlex, Semantic Scholar, PATSTAT
Embeddings	9	SPECTER2, Word2Vec, mat2vec
Social Media	6	Reddit, Twitter, Weibo, 4chan
Directories	25+	Researcher Midway paths

6. Recent Enhancements (v2)

6.1 Web Search Integration

Added Anthropic built-in web search using the web_search_20250305 tool type. Triggers on queries about "recent trends", "latest", "current", or "beyond the lab".

6.2 Enhanced Conversation Memory

Feature	Implementation	Benefit
Entity Tracking	EntityExtractor class	Track people/datasets mentioned
Recency Weighting	24hr temporal decay	Recent mentions ranked higher
Auto-Summarization	At 15 messages	Compress old context
Structured Context	Entities+Summary+Recent	Richer agent context

7. Test Results & Validation

7.1 Entity Extraction Results

Scenario	Before	After	Improvement
Honglin Bao Research	2	3	+Knowledge Lab
Reddit Data Analysis	0	3	+Reddit, Midway, Twitter
Computing Resources	1	4	+Midway, GPU, RCC
Mixed Topic	1	7	Full coverage

7.2 Memory Performance

Metric	Result	Assessment
Entity extraction	0.24ms/msg	Negligible overhead
Context generation	0.002ms/call	Instant
Memory compression	5.5x	Significant reduction

8. Performance Metrics

8.1 End-to-End Latency

Pipeline Stage	Time
-----	-----
Query Reformulation	~500ms
Semantic Search (FAISS)	<1ms
Keyword Search (BM25)	<1ms
Cross-Encoder Reranking	~800ms
-----	-----
RAG Subtotal	~1.5-2.0s
-----	-----
Claude Response	~3-5s
-----	-----
TOTAL	~5-7s

8.2 Model Usage

Query Type	Model	Latency	Example
Simple factual	Haiku	2-3s	Who is James Evans?
Complex analysis	Sonnet	4-5s	Compare X and Y research
Classification	Haiku	~500ms	Intent determination

9. Further Improvements & Roadmap

This section outlines planned enhancements and potential features for future versions of CHORUS. Items are categorized by priority and estimated complexity.

9.1 Short-Term Improvements (High Priority)

LLM-Based Query Classification

Replace regex-based classification with LLM classification for improved accuracy. Current regex patterns (75+) are brittle and require manual maintenance. An LLM classifier would better handle edge cases and ambiguous queries.

- Estimated improvement: 15-20% better intent detection
- Trade-off: +200-300ms latency per query
- Implementation: Fine-tune Haiku with classification examples

Semantic Registry Index

Add embedding-based search for registry data (people, datasets, projects). Currently, registry lookup uses exact/fuzzy string matching. Semantic search would enable queries like "researchers working on similar topics to X".

- Embed all registry entities with descriptions
- Enable similarity-based discovery
- Support "find similar" queries across entity types

Response Streaming

Implement streaming responses for both CLI and Slack interfaces to improve perceived latency. Users would see responses appear incrementally rather than waiting 5-7 seconds for complete output.

9.2 Medium-Term Enhancements

Hypergraph-Based Relevance Boosting

Use the existing hypergraph (1,312 nodes, 594 edges) to boost search relevance. When a query mentions an entity, boost results from connected entities.

- Query "James Evans" -> boost collaborators, co-authored papers
- Query "APTO project" -> boost team members, related datasets
- Leverage co-authorship and topic similarity edges

Proactive Recommendations

Based on conversation context, proactively suggest relevant resources, people, or datasets the user might not have asked about but could be useful.

- "You might also want to talk to [person] about this"
- "Related dataset available: [dataset]"
- "Similar project: [project]"

Multi-Turn Planning Agent

Add a planning specialist that can break down complex research questions into sub-queries, execute them in sequence, and synthesize results.

- Handle: "Help me find collaborators for an NLP project"
- Execute: topic search -> people search -> availability check
- Return: synthesized recommendation with reasoning

Automated Registry Updates

Implement automated weekly sync with external data sources to keep registry information current without manual intervention.

- OpenAlex API sync for publication metrics
- Midway directory crawler for resource updates
- Google Scholar scraping for citation updates

9.3 Long-Term Vision

Interactive Hypergraph Visualization

Build a web-based D3.js visualization of the research hypergraph, allowing users to explore relationships between people, projects, and publications visually. Integrated with the chat interface for drill-down queries.

Voice Interface

Add speech-to-text and text-to-speech capabilities for hands-free interaction. Useful for lab settings where users may be working at equipment.

Cross-Lab Federation

Enable CHORUS instances at different labs to share anonymized query patterns and collaborate on improving responses. Build a network of research assistants.

Research Paper Integration

Directly index and chunk lab publications for full-text search. Enable queries like "What did we find in the 2024 Science paper about X?" with direct quotes.

9.4 Technical Debt & Infrastructure

WSGI Server Upgrade

Replace Python `http.server` with `gunicorn/uvicorn` for production deployment. Current single-threaded server handles concurrent requests but is not optimal.

- Add proper process management (`systemd/supervisor`)
- Implement health checks and auto-restart
- Add request logging and monitoring

Test Coverage Expansion

Increase automated test coverage for edge cases and integration scenarios. Current tests focus on happy paths; need more adversarial testing.

- Add fuzz testing for query classification
- Integration tests for full pipeline
- Performance regression tests

Documentation & Onboarding

Create comprehensive documentation for new developers and users, including API reference, deployment guides, and troubleshooting resources.

9.5 Prioritized Roadmap

Item	Priority	Complexity	Impact
Response Streaming	High	Low	UX improvement
Semantic Registry Index	High	Medium	Better discovery
LLM Classification	High	Medium	15-20% accuracy
Hypergraph Boosting	Medium	Medium	Relevance improvement
Proactive Recommendations	Medium	Medium	User delight
Automated Registry Sync	Medium	High	Data freshness
Multi-Turn Planning	Medium	High	Complex queries
WSGI Server	Low	Low	Production readiness
Hypergraph Visualization	Low	High	Exploration UX
Voice Interface	Low	High	Accessibility

9.6 Known Limitations

Current system limitations that should be addressed in future versions:

1. Single-Threaded RAG Server

Python http.server is not optimized for high concurrency. Under heavy load, requests may queue. Stress testing shows 100% success rate for 50 concurrent queries, but production deployment should use gunicorn.

2. Sparse Hypergraph

Currently 594 edges (mostly publication edges). Missing discussion edges, computation dependencies, and mentorship relationships. LLM extraction from documents could add 5,000+ edges.

3. Static Registry Data

Lab and Midway registries require manual updates. Publication metrics become stale. Automated sync would maintain freshness.

4. Limited Multi-Language Support

Entity extraction optimized for English. International collaborator names and non-English datasets may not be recognized correctly.

5. No Offline Mode

System requires active connections to RAG server and Anthropic API. No graceful degradation for offline scenarios.

10. Conclusion

CHORUS v2.0 represents a mature, production-ready knowledge management system with sophisticated multi-agent orchestration, hybrid search, and conversation memory. The system successfully serves the Knowledge Lab community with sub-7-second response times and high accuracy on factual queries.

The roadmap outlined in Section 9 prioritizes improvements that will enhance user experience (streaming, proactive recommendations), improve accuracy (LLM classification, semantic registry), and enable new capabilities (hypergraph visualization, multi-turn planning). With continued development, CHORUS can evolve into a comprehensive research intelligence platform.